</> Software Development on Linux

26-12-2025

0. Before We Start (🧠 Mindset & Setup)

- Everything is a tool
- The terminal is your superpower
- You *own* your system

Update first. Always.

```
$ sudo apt update && sudo apt upgrade -y
```

Install the essentials:

```
$ sudo apt install -y build-essential curl wget git
software-properties-common
```

📖 Docs:

- `https://wiki.debian.org/Apt`
- `https://help.ubuntu.com/community/AptGet/Howto`

1. Compiling Suckless Tools (⚙ st, dmenu)

Suckless tools are **minimal, fast, and opinionated**. You compile them yourself. No training wheels.

Install dependencies

```
$ sudo apt install -y libx11-dev libxft-dev libxinerama-dev
```

Clone and build `st` (simple terminal)

```
$ git clone https://git.suckless.org/st
$ cd st
$ sudo make install clean
```

Same idea for `dmenu`:

```
$ git clone https://git.suckless.org/dmenu
$ cd dmenu
$ sudo make install clean
```

🔥 **Pro tip:** You *edit config.h and recompile*. That's the suckless way.

📖 **Official docs:**

- `https://suckless.org/st/`
- `https://suckless.org/dmenu/`

2. Run a Simple Server & Share Files (🖧)

Python HTTP server (the classic)

```
$ python3 -m http.server 8000
```

Now anyone on your network can access:

```
http://YOUR_IP:8000
```

⚠ Warning: http.server is not recommended for production. It only implements basic security checks.

Node.js quick server

```
$ npx serve .
```

Docs:

- https://docs.python.org/3/library/http.server.html
- https://www.npmjs.com/package/serve

---

3. PHP & XAMPP (php Yes, PHP Still Exists ☻)

Choose **One** of the following ways below

Install PHP

```
$ sudo apt install -y php php-cli php-mysql
```

Run a PHP dev server:

```
$ php -S localhost:8000
```

XAMPP (All-in-One)

Download from Apache Friends:

```
https://www.apachefriends.org/index.html
```

Install:

```
$ chmod +x xampp-linux-*-installer.run
$ sudo ./xampp-linux-*-installer.run
```

Start it:

```
$ sudo /opt/lampp/lampp start
```

Docs:

- https://www.php.net/manual/en/
- https://www.apachefriends.org/faq_linux.html

---

4. Node.js & Your First React App (⚛)

Install Node.js (LTS)

```
# Download and install nvm:
$ curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh
| bash

# in lieu of restarting the shell
$ \."$HOME/.nvm/nvm.sh"

# Download and install Node.js:
$ nvm install 24
```

Verify:

```
# Verify the Node.js version:
$ node -v # Should print "v24.12.0".

# Verify npm version:
$ npm -v # Should print "11.6.2".
```

Create a React app

```
$ npx create-react-app my-app
$ cd my-app
$ npm start
```

Boom. Browser opens. Magic.

Docs:

- https://nodejs.org/en/docs
- https://react.dev/learn

---

5. Java Basics on Linux ()

Install OpenJDK

```
$ sudo apt install default-jdk
```

You can also install specific versions of OpenJDK (e.g., Java 11 or Java 21) by searching and choosing the one you need:

```
$ apt search openjdk
$ sudo apt install openjdk-11-jdk
```

Verify:

```
$ java -version
$ javac -version
```

Hello World (because tradition)

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello Linux");
    }
}
```

Compile & run:

```
$ javac Hello.java
$ java Hello
```

Docs:

- https://openjdk.org/
- https://docs.oracle.com/javase/tutorial/

---

6. Browsers, VS Code & Arduino IDE

Browsers

```
$ sudo apt install -y firefox
```

Chrome:

```
https://www.google.com/chrome/
```

VS Code

Download from VS Code official website :

```
https://code.visualstudio.com/
```

Arduino IDE

```
$ sudo apt install -y arduino
```

Docs:

- https://code.visualstudio.com/docs/setup/linux
- https://docs.arduino.cc/

---

7. Compilers & Debugging Tools (🐞)

C/C++

```
$ sudo apt install -y gcc g++ gdb
```

Compile:

```
$ gcc main.c -o main
```

Debug:

```
$ gdb ./main
```

Docs:

- https://gcc.gnu.org/onlinedocs/
- https://www.gnu.org/software/gdb/documentation/

8. Makefiles & AVR Toolchain (</>)

Makefile basics

```
all:
    gcc main.c -o main
```

Run:

```
$ make
```

AVR toolchain

```
$ sudo apt install -y avr-libc avrdude gcc-avr
```

Helper Makefile to compile AVR code

```
#MCU name
MCU = atmega16

# CPU frequency
F_CPU = 8000000UL

# Compiler and tools
CC = avr-gcc
OBJCOPY = avr-objcopy
SIZE = avr-size
DIR = build

# Source file
SRC = main.c

# Output files
TARGET = main
ELF = $(DIR)/$(TARGET).elf
```

```
HEX = $(DIR)/$(TARGET).hex
# Compiler flags
CFLAGS = -mmcu=$(MCU) -DF_CPU=$(F_CPU) -Os

hex: $(HEX)
elf: $(ELF)

$(ELF): $(SRC)
    mkdir -p $(DIR)
    $(CC) $(CFLAGS) -o $@ $^

$(HEX): $(ELF)
    $(OBJCOPY) -O ihex -R .eeprom $< $@
    $(SIZE) $<

clean:
    rm -rf build/

.PHONY: hex elf clean
```

Used for Arduino & AVR microcontrollers.
Run:

```
$ make elf # make elf binary
$ make hex # make hex binary
$ make clean # remove build/ directory
```

Docs:

- https://www.gnu.org/software/make/manual/
- https://www.nongnu.org/avr-libc/

9. Python, pip & virtual environments (🐍)

Install Python

```
$ sudo apt install -y python3 python3-pip python3-venv
```

Virtual environment (DO THIS)

```
$ python3 -m venv venv
$ source venv/bin/activate
```

Install packages:

```
$ pip install requests bs4
```

Docs:

- https://docs.python.org/3/tutorial/venv.html
- https://pip.pypa.io/en/stable/

---

10. AppImage & Desktop Shortcuts (🖵)

Run AppImage:

```
$ chmod +x MyApp.AppImage
$ ./MyApp.AppImage
```

Desktop shortcut

Create:

```
$ ~/.local/share/applications/myapp.desktop
```

```
[Desktop Entry]
Name=My App
Exec=/path/MyApp.AppImage
Type=Application
Icon=/path/icon.png
```

Docs:

- https://docs.appimage.org/

Helping Tutorial:

- mmbesar

11. Update System & Change Repositories

Edit sources:

```
$ sudo nano /etc/apt/sources.list
```

Update:

```
$ sudo apt update && sudo apt upgrade
```

Docs:

- https://wiki.debian.org/SourcesList
- https://help.ubuntu.com/community/Repositories

12. Kali Repositories (⚠ Read This)

> ⚠ **Don't mix Kali repos with Ubuntu casually.**
> You *will* break things.

Safe use case: **specific tools only**.

```
deb http://http.kali.org/kali kali-rolling main non-free contrib
```

Use pinning if you *must*.

Docs:

- `kali-linux-sources-list-repositories [official]`

Helping Tutorial:

- `Adding Kali linux repos to debian distros`

13. Distrobox (Run Any Distro Safely)

```
$ sudo apt install -y podman
```

Create container:

```
$ distrobox create -n arch --image archlinux:latest
$ distrobox enter arch
```

Now you're in another distro. No VM. No pain.

Docs:

- `https://distrobox.it/`

14. Docker & Containers

Install Docker

```
$ sudo apt install -y docker.io
$ sudo usermod -aG docker $USER
$ newgrp docker
```

Test:

```
$ docker run hello-world
```

Run a container:

```
$ docker run -it ubuntu bash
```

Docs:

- https://docs.docker.com/engine/install/ubuntu/

Helping Tutorial:

- Home Server - Install and Use Docker